MODELOS TENSORFLOW

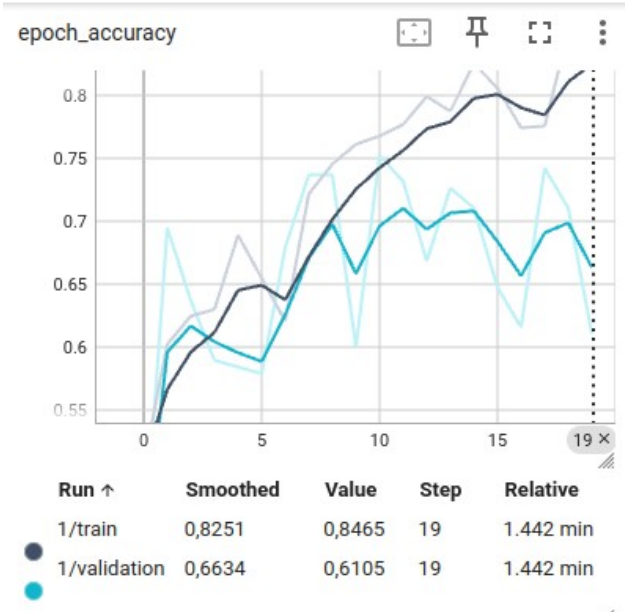
Dado que el espacio del que disponemos en la raspberry es muy limitado, vamos a tratar de utilizar modelos pequeños que tengan la mayor tasa de acierto dentro de lo posible.
Debemos de tener en cuenta que el tamaño del modelos únicamente me va a depender de la resolución de las imágenes, del número de capas y de neuronas de mi red neuronal. El número de imágenes que utilice para entrenarle no me afecta, por lo tanto, usaré un dataset de casi mil imágenes (La mitad incluyen un caracol y la otra mitad no).
Para ahorrar mucho espacio, reduciremos la resolución de las fotos a 100x100 y en lugar de emplear imágenes a color, las usaremos en escala de grises, de tal forma que cada pixel solo podrá tomar valores de 0 a 255.
En cuanto al modelo empleado, debemos de tener en cuenta que después queremos convertirlo en TensorFlow Lite por lo que es conveniente utilizar modelos sencillos que cuenten con lo más básico. Por lo tanto, la idea inicial es probar modelos sencillos e ir modificándolos a mano hasta llegar a la solución óptima.

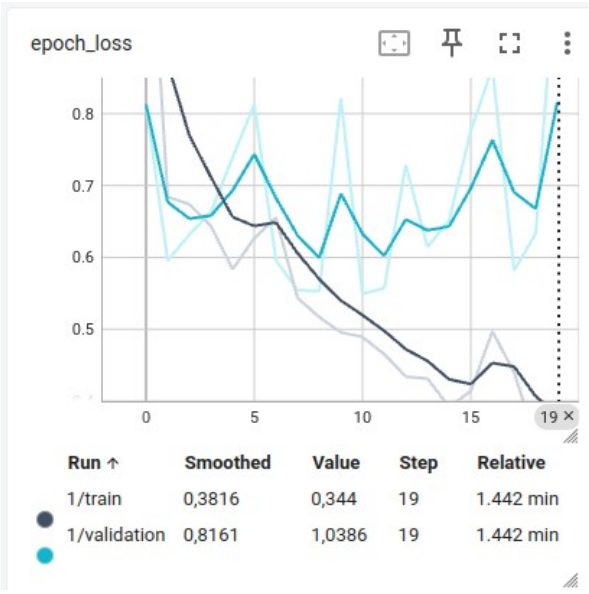
COMPARACIÓN DE MODELOS

Para comparar los modelos, tomaremos medidas de:
- Exactitud/accuracy: Que indica el porcentaje de aciertos que ha tenido el modelo.
- Pérdida/loss: Indica cuanto de cerca está de obtener las predicción correcta.

Cuanto mejor sea nuestra exactitud y menor sea nuestra pérdida, mejor será nuestro modelo. Por lo tanto, trataremos de que la exactitud se acerque los máximo posible a 1 y que la pérdida se aproxime a 0.

MODELO 1 (DENSO)





En este modelo cada neurona se conecta con todas, esto no es bueno porque este tipo de neuronas no entienden el espacio y solo ven una lista de números sin orden.

Se puede observar claramente en el gráfico que, durante el entrenamiento, el modelo está memorizando muy bien las imágenes pero cuando ve las imágenes de evaluación se equivoca. Por otro lado, si analizamos la pérdida, durante el entrenamiento se ve cómo va bajando progresivamente mientras que en la validación ocurre casi que lo contrario, demostrando así que está muy lejos de obtener la respuesta esperada.

MODELO 2 (CNN)

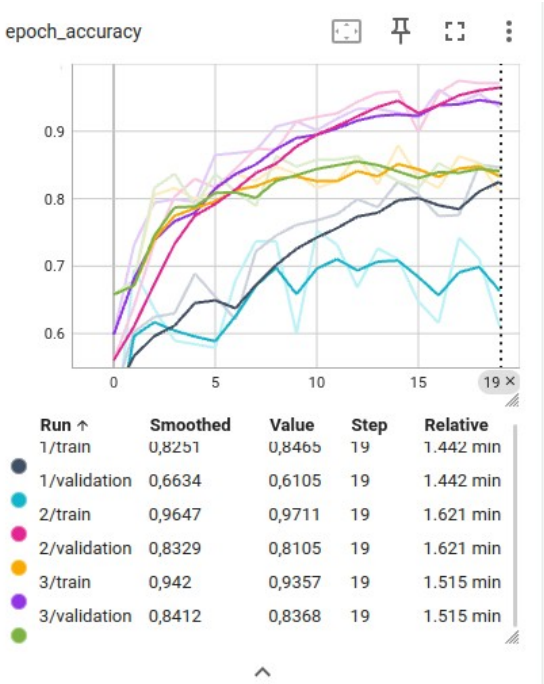
Con respecto al modelo anterior, podemos ver una notable mejora, esto es porque se utilizan capas convolucionales y de reducción, estas son capas que funcionan mejor para el aprendizaje de imágenes dado que se encargan de buscar patrones que les pueda servir para decidir.

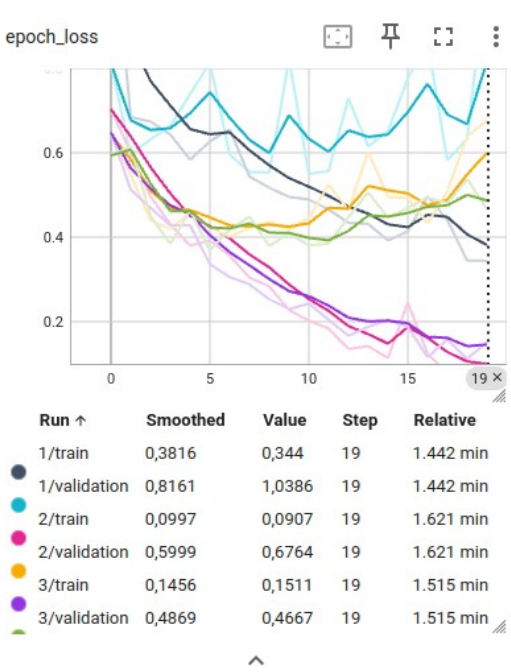
Cómo podemos observar, la exactitud durante el entrenamiento se acerca mucho a 1. Esto quiere decir que está aprendiendo mucho, pero a la hora de la verdad, en la validación demuestra que sigue memorizando las imágenes. En la pérdida, podemos observar que, aunque en el entrenamiento llegue a ser casi 0, en la realidad observamos que no baja del 0.4 y que en lugar de ir aproximándose cada vez más a 0, se vuelve a alejar.

MODELO 3 (CNN + Dropout)

Este modelo quizás sea el que mejor funcione, sin embargo, tampoco es uno muy bueno. Esta pequeña mejora se debe a la introducción de una capa de dropout, la cual se encarga de desactivar neuronas aleatorias para que la red sea capaz de identificar distintas características que le ayuden a distinguir las imágenes.

COMPARACIÓN COMPLETA





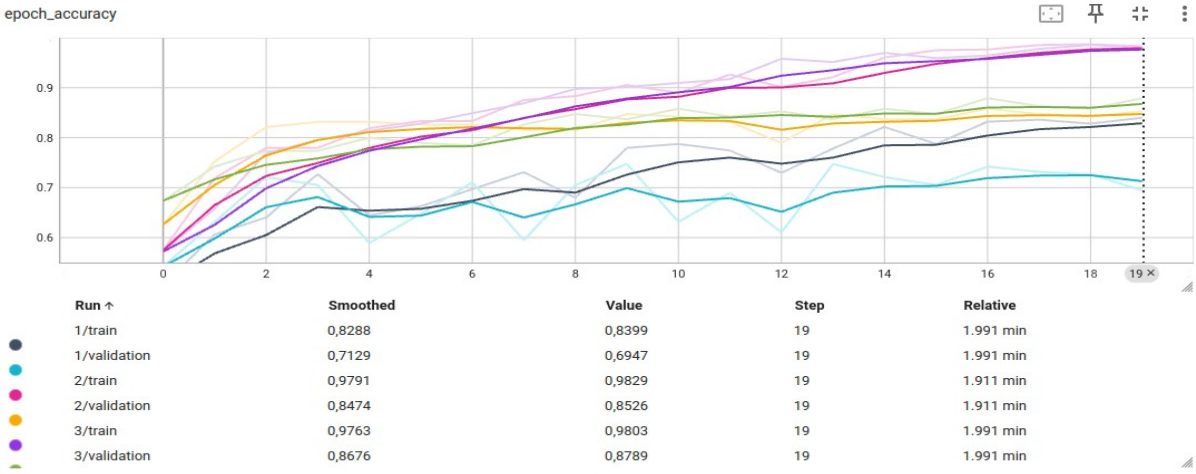
Es cierto que hemos conseguido generar un modelo que sea capaz de distinguir un caracol con alrededor de 80% de exactitud, pero hay más formas de hacer que esta cifra aumente, las probaremos a continuación.

AUMENTO DE DATOS

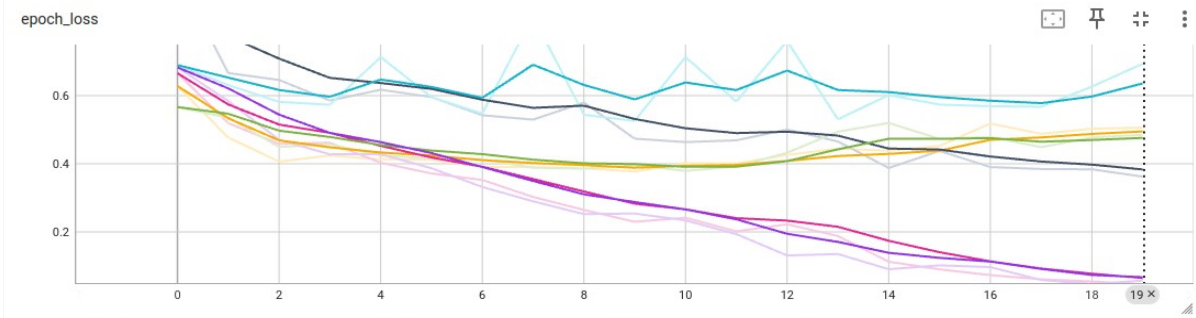
Existe una forma de entrenar mejor al modelo, que consiste en el “Aumento de Datos” donde se modifica el zoom de las imágenes, la rotación, el volteo (flip) o el brillo. De esta forma trataremos de evitar que el modelo memorice ciertos patrones de las imágenes que no son identificativos, por ejemplo, me he dado cuenta de que la mayoría de las imágenes los caracoles salen mirando hacia la derecha y el modelo podría asumir que, mirar hacia la derecha es un rasgo característico del caracol y por consecuencia, si le pregunto por una foto de un caracol mirando hacia la izquierda, me diga que no es un caracol.

También probaré a subir un poco más épocas por si me quedo corta tras el aumento de datos ya que con el aumento de datos, no va a llegar tan rápido el momento en el que deje de “aprender”.

Tras estos cambios obtenemos los siguientes gráficos:



Run	Smoothed	Value	Step	Relative
1/train	0,8288	0,8399	19	1.991 min
1/validation	0,7129	0,6947	19	1.991 min
2/train	0,9791	0,9829	19	1.911 min
2/validation	0,8474	0,8526	19	1.911 min
3/train	0,9763	0,9803	19	1.991 min
3/validation	0,8676	0,8789	19	1.991 min



Run	Smoothed	Value	Step	Relative
1/train	0,3834	0,3623	19	1.991 min
1/validation	0,6358	0,6935	19	1.991 min
2/train	0,0648	0,0458	19	1.911 min
2/validation	0,495	0,5062	19	1.911 min
3/train	0,0671	0,0571	19	1.991 min
3/validation	0,4759	0,485	19	1.991 min

OTROS MODELOS

Otra opción para contemplar, es tomar un modelo de repositorios de Google o similar y entrenarlo con mi dataset. De esta forma lo más seguro es que la tasa de acierto sea mayor al 90%. Sin embargo, tenemos la restricción de que estos modelos ocupan bastante más espacio que uno propio. En un futuro habría que averiguar si se pudieran usar en la raspberry. (https://www.tensorflow.org/hub?hl=es-419 , aquí podemos encontrar muchos modelos pre-entrenados que nos podrían resultar útiles).